AURA-FACE: Enterprise Biometric Face Attendance & Anti-Spoofing Verification System

1. Executive Summary

The AURA-FACE system provides a robust, high-speed, and secure biometric attendance tracking solution. The primary problem solved is the prevention of buddy-punching and fraudulent attendance entries using sophisticated L2-normalized deep embeddings combined with real-time MediaPipe liveness verification. The system integrates a sleek CustomTkinter UI for kiosk operation, enabling sub-millisecond retrieval via Meta's FAISS, ensuring a frictionless and secure experience for both administrators and employees.

2. Comprehensive Tech Stack & Architectural Justifications

Layer	Technology	Justification / Role
Presentation Layer	CustomTkinter	Modern dark-mode Kiosk & Admin Dashboard.
Face Detection	OpenCV YuNet	Lightweight ONNX, high-speed bounding box & landmark extraction.
Feature Extraction	MobileNetV2 (PyTorch)	Produces 128-dimensional unit-normalized embedding vectors.
Vector Search	Meta FAISS	IndexIDMap2 wrapping IndexFlatL2 for sub-millisecond similarity matching.
Anti-Spoofing / Liveness	MediaPipe Face Mesh	Tracks 468 landmarks to calculate Eye Aspect Ratio (EAR) for blink detectio
Relational Storage	SQLite3	attendance.db tracks users, relationships, and timestamped attendance logs

3. Module-by-Module Code Architecture (File-by-File Breakdown)

face_engine.py

Initializes the FaceEngine using YuNet for fast face detection. A PyTorch-based MobileNetV2 feature extractor processes the aligned face crops to output a 128-d vector. L2 normalization is strictly applied at the model output to map faces to a unit hypersphere.

liveness_detector.py

Prevents spoofing by verifying blinks. It calculates the Eye Aspect Ratio (EAR) using the formula: EAR = (||p2-p6|| + ||p3-p5||) / (2 * ||p1-p4||). State machine triggers a valid live read if EAR drops below 0.20 (closed) and rises above 0.25 (open).

vector_index.py

Manages the FAISS indexing. It creates an IndexIDMap2 wrapper around IndexFlatL2 to map user IDs (int64) to embeddings. Methods include add_vector(uid, vec), search(vec, threshold) for top-1 mapping, and remove_vector(uid) leveraging faiss.IDSelectorArray.

db_manager.py

Handles SQLite3 schema setups for the users and attendance tables. Supports full cascading deletes, sequential SQLite autoincrements, and enforces foreign-key logic mapping numeric IDs to string-based User Codes.

app.py

The core presentation tier. Provides a dual-mode workflow: A Kiosk scanner loop for live OpenCV inference, and a secure Admin Management Portal. Manages threading, UI updates via CustomTkinter, real-time bounding boxes, auto-close timers, and table renderings.

evaluate_accuracy.py & Maintenance Tools

Tools to benchmark the system via FAR, FRR, Precision, and Recall metrics. Maintenance scripts such as resequence_uids.py ensure database and FAISS index harmony over time.

4. End-to-End Operational Workflows

Enrollment Workflow

1. Photo capture triggered in UI.
2. YuNet crop extracts the face.
3. Embedder generates a 128-d L2-normalized vector.
4. Strict duplicate check (Threshold: 0.20) runs against FAISS.
5. SQLite records the User ID.
6. FAISS indexes the vector associated with the SQLite ID.

Attendance Kiosk Workflow

1. Live camera feed processed continuously.
2. YuNet detects faces.
3. Liveness blink verification ensures physical presence.
4. MobileNetV2 embedding is generated.
5. FAISS searches the index for a match (Threshold: 0.55).
6. SQLite logs the attendance timestamp.
7. 1.5s visual confirmation displayed before camera auto-shutoff.